Physical Design – Terminology Concepts Reference

Database Systems

Based on Lecture 8 by Tiantian Liu

Fall 2025

Contents

1 Quick Reference Guide

Essential Concepts - At a Glance

Storage Hierarchy:

- **Volatile**: Cache, Main Memory (lost when power off)
- **Non-volatile**: Flash, Magnetic Disk, Optical Disk, Tape (persists)
- **Trade-off**: Speed vs Cost vs Persistence

File Organization:

- **Heap**: Records placed anywhere with free space
- **Sequential**: Records ordered by search key
- **B+-tree**: Balanced tree, supports efficient insert/delete
- **Hashing**: Hash function determines bucket location

B+-Tree Properties (degree n):

- Internal nodes: between $\lceil n/2 \rceil$ and n children
- Leaf nodes: between $\lceil (n-1)/2 \rceil$ and $n-1$ values
- Root: at least 2 children (if not a leaf)
- All leaf nodes at same level

Indexing:

- **Primary (Clustering)**: Index on sequentially ordered file
- **Secondary (Nonclustering)**: Index on different order
- **Dense**: Index entry for every search-key value
- **Sparse**: Index entries for some search-key values only

2 Physical Storage Media

Storage Classification

Volatile Storage:

- **Definition**: Contents lost when power is switched off
- **Examples**: Cache, Main Memory (RAM)
- **Characteristics**: Very fast, expensive per unit

Non-volatile Storage:

- **Definition**: Contents persist even when power is switched off
- **Examples**: Flash memory, Magnetic disk, Optical disk, Tape

- **Characteristics**: Slower, cheaper per unit, persistent

Key Factors:

- **Speed**: How fast data can be accessed
- **Cost**: Price per unit of data
- **Reliability**: How dependable the storage is

Storage Hierarchy

From fastest/most expensive to slowest/least expensive:

1. **Cache** (Volatile)
   - Fastest, most expensive
   - Small capacity (KB to MB)
2. **Main Memory (RAM)** (Volatile)
   - Very fast, expensive
   - Medium capacity (GB)
3. **Flash Memory** (Non-volatile)
   - Fast, moderate cost
   - Increasing capacity (GB to TB)
4. **Magnetic Disk** (Non-volatile)
   - Moderate speed, low cost
   - Large capacity (TB)
   - Primary storage for databases
5. **Optical Disk** (Non-volatile)
   - Slower, very low cost
   - Archival storage
6. **Tape** (Non-volatile)
   - Slowest, cheapest
   - Backup and archival

2.1 Magnetic Disk Structure

Disk Components

Platter:

- Circular disk coated with magnetic material
- Divided into circular **tracks**

- Typical disk: 50K-100K tracks per platter

Track:

- Circular path on platter
- Divided into **sectors**

Sector:

- **Smallest unit** of data that can be read or written
- Typical size: 512 bytes

Block (Page):

- Contiguous sequence of sectors from a single track
- **Basic unit** of data transfer between disk and memory
- Typical size: 4KB, 8KB, or larger

Read-Write Head:

- Reads/writes data on platter surface
- Moves radially across tracks

3 File Organization

File Organization Overview

Definition: How records are arranged and stored in files.
Database Storage:

- Database = collection of **files**
- File = sequence of **records**
- Record = sequence of **fields**

Key Requirements:

- Process records sequentially
- Efficient key-value search
- Fast insertion/deletion of records

Performance Objectives:

- Minimize wasted space
- Fast response time
- High transaction throughput

3.1 Fixed-Length Records

Fixed-Length Record Storage

Assumption:

- Record size is fixed
- Each file has records of one type only
- Records are smaller than a disk block

Simple Approach:

- Store record i starting from byte $n \times i$ where n = record size
- **Problem**: Records may cross block boundaries

Better Approach:

- Do not allow records to cross block boundaries
- May waste some space at end of blocks
- Simplifies record access

Deletion in Fixed-Length Records

Three Alternatives for Deleting Record i:
Option 1: Shift Records

- Move records $i+1, ..., n$ to positions $i, ..., n-1$
- **Advantage**: No wasted space
- **Disadvantage**: Expensive (many moves)

Option 2: Move Last Record

- Move record n to position i
- **Advantage**: Only one move
- **Disadvantage**: Changes record order

Option 3: Free List (Best)

- Do not move records
- Link all free records on a **free list**
- Store pointer to first free record in file header
- **Advantage**: Fast deletion and insertion
- **Disadvantage**: Small space overhead for pointers

3.2 Variable-Length Records

When Variable-Length Records Occur

Scenarios:

- Storage of multiple record types in a file
- Record types with variable-length fields (e.g., `varchar`)
- Record types with repeating fields
- Record types with optional fields

Variable-Length Record Representation

Structure:

- Fixed-length attributes stored first
- Variable-length attributes represented by:
  - (offset, length) pairs in fixed-length portion
  - Actual data stored after all fixed-length attributes
- **Null bitmap**: Indicates which attributes are null

Variable-Length Record Example

Schema:

```
create table instructor (
    ID          varchar(10),
    name        varchar(20),
    dept_name   varchar(20),
    salary      int(8)
);
```

Record: (10101, Srinivasan, Comp. Sci., 65000)
Storage Layout:

- Bytes 0-3: Null bitmap (0000) - no nulls
- Bytes 4-7: salary (65000) - fixed-length int
- Bytes 8-11: (offset=26, length=10) for ID
- Bytes 12-15: (offset=21, length=5) for name
- Bytes 16-19: (offset=36, length=9) for dept_name
- Byte 21-25: "10101" (ID value)
- Byte 26-35: "Srinivasan" (name value)
- Byte 36-44: "Comp. Sci." (dept_name value)

3.3 File Organization Types

Heap File Organization

Definition: Record can be placed anywhere in the file where there is space.
Characteristics:

- **Insertion**: Very fast - add to end or any free space
- **Search**: Slow - must scan entire file
- **Deletion**: Fast - mark as deleted or use free list

Use Case: When records are typically accessed via an index, not sequentially.

Sequential File Organization

Definition: Records stored in sequential order based on search key.
Characteristics:

- Records ordered by a **search key**
- Suitable for sequential processing
- Efficient for range queries

Insertion:

- Locate position where record should be inserted
- If free space exists, insert there
- Otherwise, insert in **overflow block**
- Update pointer chain

Deletion:

- Use pointer chains
- Mark record as deleted
- Update pointers to maintain sequential order

Problem: Frequent insertions/deletions degrade performance (many overflow blocks).
Solution: Periodically reorganize file to restore sequential order.

Multitable Clustering File Organization

Definition: Store records of several different relations in one file.
Motivation: Store related records on same block to minimize I/O.
Example:

- Store `department` and `instructor` records together
- Group instructors with their department on same block

Benefits:

- Efficient for queries that join these relations
- Reduces I/O by co-locating related data

Drawbacks:

- Poor performance for queries on individual relations
- More complex to maintain

4 Indexing

Index Basics

Purpose: Speed up access to desired data (like a book index).
Search Key:

- Attribute or set of attributes used to look up records
- Does NOT have to be a primary key or unique

Index File:

- Consists of **index entries** of the form:

$$\text{(search-key, pointer)}$$

- Much smaller than data file
- Pointer references the actual data record or block

4.1 Types of Indexes

Primary vs Secondary Index

Primary Index (Clustering Index):

- Index whose search key specifies the **sequential order** of the file
- File is physically ordered by this key
- Also called **clustering index**
- Search key usually (but not necessarily) the primary key

Secondary Index (Nonclustering Index):

- Index whose search key specifies an order **different from** sequential order
- File is NOT ordered by this key
- Also called **nonclustering index**

Note: A table can have only ONE primary index but MULTIPLE secondary indexes.

Dense vs Sparse Index

Dense Index:

- Index entry appears for **every** search-key value in the file
- Points directly to the record (or first record with that key)
- **Required** for secondary indexes
- **Advantage**: Faster lookup
- **Disadvantage**: Larger index size

Sparse Index:

- Contains index entries for **only some** search-key values
- **Applicable ONLY** when records are sequentially ordered on search key
- To locate record with search-key value K:
  1. Find index entry with largest search-key value $< K$
  2. Search file sequentially from that record
- **Advantage**: Smaller index size
- **Disadvantage**: Slower lookup (requires sequential search)

Trade-off:

- Dense: Faster location, larger index
- Sparse: Smaller index, slower location

Primary Dense Index

For `instructor` file sorted on `ID`:
Dense index on ID:

ID	Pointer
10101	$\rightarrow$ record
12121	$\rightarrow$ record
15151	$\rightarrow$ record
22222	$\rightarrow$ record
...	...

Every search-key value has an index entry.

Primary Sparse Index

For `instructor` file sorted on `dept_name`:
Sparse index on dept_name:

dept_name	Pointer
Biology	→ first Biology record
Comp. Sci.	→ first Comp. Sci. record
Finance	→ first Finance record
...	...

Only some search-key values have index entries.
To find "Elec. Eng." department:

1. Find index entry for largest key < "Elec. Eng." (which is "Comp. Sci.")
2. Follow pointer to first Comp. Sci. record
3. Scan sequentially until finding Elec. Eng.

Secondary Index

For `instructor` file sorted on `ID`, create secondary index on `salary`:
Secondary index on salary:

- Must be **dense** (file not ordered by salary)
- Index entry points to bucket containing pointers to all records with that salary

salary	Pointer
45000	→ bucket of record pointers
50000	→ bucket of record pointers
65000	→ bucket of record pointers
...	...

5 B+-Tree

B+-Tree Overview

What is a B+-Tree?

- **Balanced tree** index structure
- Most widely used index structure in databases
- Supports efficient insertion, deletion, and search
- Automatically reorganizes with small local changes

Why B+-Tree?

- Maintains sorted data
- Allows searches, insertions, deletions in $O(\log n)$ time
- All leaf nodes at same level (balanced)
- Efficient for both point queries and range queries

5.1 B+-Tree Structure

B+-Tree Node Structure

Typical Node:

$$P_1 \quad K_1 \quad P_2 \quad K_2 \quad ... \quad P_{n-1} \quad K_{n-1} \quad P_n$$

Components:

- K_i: Search-key values (ordered: $K_1 < K_2 < ... < K_{n-1}$)
- P_i: Pointers
  - In **non-leaf nodes**: pointers to children
  - In **leaf nodes**: pointers to records or buckets

Search-Key Ordering:

- Initially assume no duplicate keys
- Keys in a node: $K_1 < K_2 < K_3 < ... < K_{n-1}$

B+-Tree Properties

A B+-tree with degree n (fanout) is a rooted tree satisfying:

1. **All paths from root to leaf have same length**
   - **Plain English**: Tree is perfectly balanced
2. **Each internal node has between $\lceil n/2 \rceil$ and n children**
   - **Plain English**: Internal nodes are at least half full
   - Example: If $n = 6$, internal nodes have 3-6 children
3. **Each leaf node has between $\lceil (n-1)/2 \rceil$ and $n-1$ values**
   - **Plain English**: Leaf nodes are at least half full
   - Example: If $n = 6$, leaf nodes have 3-5 values
4. **Root is special case:**
   - If root is not a leaf, it has at least 2 children
   - If root is a leaf, it can have 1 to $n-1$ values

B+-Tree Example - Degree n

Properties for n = 4:

- Internal nodes: $\lceil 4/2 \rceil = 2$ to 4 children
- Leaf nodes: $\lceil 3/2 \rceil = 2$ to 3 values
- Root: at least 2 children (if not leaf)

Underflow Conditions:

- Leaf node underflows when keys < 2
- Internal node underflows when pointers < 2

Key Rules for B+-Tree Navigation

In a non-leaf node with key K:

- **Left pointer** of K: leads to keys $< K$
- **Right pointer** of K: leads to keys $\geq K$

Example:

$$P_1 \quad 20 \quad P_2$$

- P_1 points to subtree with keys < 20
- P_2 points to subtree with keys ≥ 20

5.2 B+-Tree Operations

B+-Tree Search

Search for value V:
Algorithm:

1. Start at root
2. At each non-leaf node:
   - Find smallest key $K_i \geq V$
   - Follow pointer P_i (pointer to right of K_{i-1} or left of K_i)
   - If no such K_i exists, follow rightmost pointer
3. At leaf node:
   - Search for V among the keys
   - Follow pointer to retrieve record

Range Query (all values $\geq V$):

1. Search for V to reach appropriate leaf
2. Follow leaf-node pointers to scan remaining leaves

B+-Tree Insertion Algorithm

High-Level Algorithm:

1. Find correct leaf node for new entry
2. Insert entry into leaf node
3. If leaf has space: done
4. If leaf is full: **split** the leaf node

Recursive Procedure:

- **If non-full node**: Insert the entry
- **If full node**:
  - Split node into two
  - For **leaf node split**: **Copy up** middle key to parent
  - For **internal node split**: **Push up** middle key to parent
  - Recursively insert middle key into parent

Important Distinction:

- **Leaf split**: Middle key is **copied up** (stays in leaf)
- **Internal split**: Middle key is **pushed up** (moves to parent)

B+-Tree Insertion Example - Insert 16* (n

Initial tree:

10	20	30

1*	5*	12*	15*	17*	21*	24*	34*	35*	38*

Step 1: Go to correct leaf (12*, 15*, 17*)
Step 2: Try to insert 16* ⇒ Leaf becomes (12*, 15*, 16*, 17*) - FULL!
Step 3: Split the leaf:

- First half: (12*, 15*)
- Second half: (16*, 17*)
- Middle key: 16 ⇒ **Copy up** to parent

Step 4: Insert 16 into parent ⇒ Parent becomes (10, 16, 20, 30) - FULL!
Step 5: Split the parent (internal node):

- First half: (10, 16)
- Second half: (30)
- Middle key: 20 ⇒ **Push up** to create new root

Result: New root with key 20, two internal nodes below.

B+-Tree Deletion Algorithm

High-Level Algorithm:

1. Find and delete entry from leaf node
2. If leaf has enough entries: done
3. If leaf underflows: **redistribute** or **merge**

Handling Underflow:

- **Redistribution**: Borrow entry from sibling (if sibling has extra)
  - For leaf nodes: Copy new separator to parent
  - Adjust keys in parent
- **Merge**: Combine with sibling (if sibling has minimum entries)
  - For leaf nodes: Combine entries, remove separator from parent
  - For internal nodes: Pull down separator from parent, then combine
  - May propagate merge to parent level

Special Case - Root:

- If root has only one child after merge, make that child the new root
- Tree height decreases by one

B+-Tree Deletion with Redistribution (n

Delete 24* from:

10	20	30

12*	15*	17*	21*	24*	34*	35*	38*

Step 1: Delete 24* from leaf ⇒ (21*) - UNDERFLOW! (needs ≥ 2)
Step 2: Check sibling (34*, 35*, 38*) - has 3 entries (can spare one)
Step 3: **Redistribute**:

- Borrow 34* from right sibling
- Left leaf becomes: (21*, 34*)
- Right leaf becomes: (35*, 38*)
- Update parent separator from 30 to 35

Result:

10	20	35

12*	15*	17*	21*	34*	35*	38*

B+-Tree Deletion with Merge (n

Delete 38* from:

10	20	35

12*	15*	17*	21*	34*	35*	38*

Step 1: Delete 38* ⇒ (35*) - UNDERFLOW!
Step 2: Check sibling (21*, 34*) - has exactly 2 entries (minimum, can't spare)
Step 3: **Merge** with left sibling:

- Combine: (21*, 34*, 35*)
- Remove separator 35 from parent
- Parent becomes: (10, 20)

Result:

10	20

12*	15*	17*	21*	34*	35*

6 Hashing

Static Hashing Basics

Key Concepts:

- **Bucket**: Unit of storage containing one or more entries
  - Typically a disk block
  - Stores records or pointers to records
- **Hash Function** h: Maps search-key values to bucket addresses
  - $h : K \to B$ where K = all search keys, B = all bucket addresses
  - Used for access, insertion, and deletion

Properties:

- Entries with different keys may map to same bucket (collision)
- Must search entire bucket sequentially to locate entry
- Goal: Uniform distribution of keys across buckets

Hash Function Example

Scenario:

- Hash `instructor` file using `dept_name` as key
- 10 buckets (numbered 0-9)
- Hash function: Sum of binary representations of characters $\mod 10$

Examples:

- $h(\text{Music}) = 1$
- $h(\text{History}) = 2$
- $h(\text{Physics}) = 3$
- $h(\text{Elec. Eng.}) = 3$ (collision with Physics!)

Result:

- Physics and Elec. Eng. records stored in bucket 3
- Must search bucket 3 sequentially to find specific department

Bucket Overflow

Causes of Overflow:

- **Insufficient buckets**: Not enough buckets for data size
- **Skew in distribution**:

  - Multiple records with same search-key value
  - Hash function produces non-uniform distribution

Handling Overflow - Overflow Chaining:

- Overflow buckets chained together in linked list
- When bucket fills, allocate overflow bucket
- Link overflow bucket to original bucket
- Also called **closed addressing** or **open hashing**

Problem:

- Long overflow chains degrade performance
- Degrades to sequential search in worst case

7 Step-by-Step Guide to Physical Design Problems

7.1 How to Solve B+-Tree Insert Problems

B+-Tree Insertion Procedure

Given: B+-tree of degree n, value to insert
Step 1: Navigate to correct leaf

1. Start at root
2. At each non-leaf, find appropriate child pointer
3. Continue until reaching leaf node

Step 2: Try to insert

- Insert value in sorted order in leaf
- If leaf has $\leq n - 1$ values: DONE
- If leaf has n values: OVERFLOW - go to Step 3

Step 3: Split leaf node

- Divide n values into two nodes:
  - First $\lceil n/2 \rceil$ values in left node
  - Remaining values in right node
- **Copy up** smallest key from right node to parent
- Update pointers

Step 4: Insert into parent

- Insert copied-up key into parent
- If parent has $< n$ pointers: DONE

- If parent has $n + 1$ pointers: OVERFLOW - go to Step 5

Step 5: Split internal node

- Divide n pointers into two nodes
- **Push up** middle key to parent (doesn't stay in either child)
- Recursively insert into parent (repeat Step 4)

Step 6: Handle root split

- If root splits, create new root with middle key
- Tree height increases by one

7.2 How to Solve B+-Tree Delete Problems

B+-Tree Deletion Procedure

Given: B+-tree of degree n, value to delete
Step 1: Navigate and delete

1. Find leaf containing value
2. Delete value from leaf
3. If leaf has $\geq \lceil (n-1)/2 \rceil$ values: DONE
4. If leaf has $< \lceil (n-1)/2 \rceil$ values: UNDERFLOW - go to Step 2

Step 2: Check sibling

- Look at immediate sibling (left or right)
- If sibling has $> \lceil (n-1)/2 \rceil$ values: go to Step 3 (Redistribute)
- If sibling has exactly $\lceil (n-1)/2 \rceil$ values: go to Step 4 (Merge)

Step 3: Redistribute (borrow from sibling)

- For **leaf nodes**:
  - Move entry from sibling to underflowing node
  - Update separator key in parent to new boundary value
  - DONE

Step 4: Merge with sibling

- For **leaf nodes**:
  - Combine entries from both nodes
  - Delete separator key from parent
  - Update pointers
- If parent now underflows (has $< \lceil n/2 \rceil$ pointers):

  - Recursively fix parent (merge or redistribute at internal level)

Step 5: Handle root

- If root has only one child:
  - Make that child the new root
  - Tree height decreases by one

7.3 Quick Formulas for B+-Tree (degree n)

Essential B+-Tree Formulas

Internal Node Constraints:

- Minimum children: $\lceil n/2 \rceil$
- Maximum children: n
- Minimum keys: $\lceil n/2 \rceil - 1$
- Maximum keys: $n - 1$

Leaf Node Constraints:

- Minimum values: $\lceil (n-1)/2 \rceil$
- Maximum values: $n - 1$

Underflow Conditions:

- Leaf underflows when: values $< \lceil (n-1)/2 \rceil$
- Internal underflows when: children $< \lceil n/2 \rceil$

Examples for n = 4:

- Internal: $\lceil 4/2 \rceil = 2$ to 4 children
- Leaf: $\lceil 3/2 \rceil = 2$ to 3 values
- Leaf underflows when values < 2
- Internal underflows when children < 2

Examples for n = 5:

- Internal: $\lceil 5/2 \rceil = 3$ to 5 children
- Leaf: $\lceil 4/2 \rceil = 2$ to 4 values
- Leaf underflows when values < 2
- Internal underflows when children < 3

8 Summary - Key Points for Exam

Essential Concepts

Storage:

- Volatile: Cache, RAM (lost on power off)
- Non-volatile: Flash, Disk, Tape (persists)
- Block (Page): Basic unit of disk-memory transfer

Records:

- Fixed-length: Use free list for deletions
- Variable-length: (offset, length) pairs + null bitmap

File Organization:

- Heap: Fast insert, slow search
- Sequential: Ordered by key, efficient for range queries
- B+-tree: Balanced, supports dynamic insert/delete
- Hashing: Direct access via hash function

B+-Tree (degree n):

- Internal: $\lceil n/2 \rceil$ to n children
- Leaf: $\lceil (n-1)/2 \rceil$ to $n-1$ values
- Insert: Split on overflow, copy/push up middle key
- Delete: Redistribute or merge on underflow

Indexes:

- Primary: File ordered by index key
- Secondary: File not ordered by index key (must be dense)
- Dense: Entry for every key
- Sparse: Entries for some keys (only for primary)

Hashing:

- Hash function maps keys to buckets
- Overflow chaining handles collisions
- Trade-off: Fast access vs bucket overflow

Common Exam Question Types

Type 1: B+-Tree Insert

1. Navigate to correct leaf
2. Insert value in sorted order
3. If overflow: split and copy/push up middle key
4. Recursively handle parent overflow

Type 2: B+-Tree Delete

1. Find and delete value
2. If underflow: check sibling
3. Redistribute (if sibling has extra) or merge
4. Recursively handle parent underflow

Type 3: Determine Index Type

- Primary vs Secondary: Is file ordered by this key?
- Dense vs Sparse: Entry for every key?

Type 4: Calculate B+-Tree Properties

- Given degree n, calculate min/max children and values
- Use formulas: $\lceil n/2 \rceil$ and $\lceil (n-1)/2 \rceil$

Type 5: Hash Function Application

- Apply given hash function to keys
- Identify bucket assignments
- Recognize collisions

Tips for Success

B+-Tree Operations:

- Always draw the tree step-by-step
- Remember: Copy up for leaf split, Push up for internal split
- Check underflow/overflow conditions carefully
- For redistribution: update parent separator keys

Common Mistakes to Avoid:

- Forgetting to update parent pointers after split/merge
- Confusing copy up (leaf) vs push up (internal)
- Miscalculating $\lceil n/2 \rceil$ (e.g., for $n = 4$, it's 2, not 2.5)

- Using sparse index for secondary index (must be dense!)

Study Strategy:

- Practice B+-tree insert/delete with different degree values
- Understand when to redistribute vs merge
- Memorize formulas for min/max children and values
- Review differences between index types

9 Practice Exercises

9.1 Exercise 1: B+-Tree Properties

Practice - Understanding B+-Tree Constraints

Problem 1: For a B+-tree with degree $n = 6$:

1. What is the minimum number of children for an internal node?
2. What is the maximum number of children for an internal node?
3. What is the minimum number of values for a leaf node?
4. What is the maximum number of values for a leaf node?
5. When does a leaf node underflow?
6. When does an internal node underflow?

Problem 2: For a B+-tree with degree $n = 7$:

1. Calculate the minimum and maximum children for internal nodes
2. Calculate the minimum and maximum values for leaf nodes
3. If a leaf node has 2 values, is it underflowing?
4. If an internal node has 3 children, is it underflowing?

Formulas to use:

- Internal: $\lceil n/2 \rceil$ to n children
- Leaf: $\lceil (n-1)/2 \rceil$ to $n-1$ values

Solutions:
Problem 1: For n = 6

1. Minimum children for internal: $\lceil 6/2 \rceil = 3$
2. Maximum children for internal: 6
3. Minimum values for leaf: $\lceil (6-1)/2 \rceil = \lceil 5/2 \rceil = 3$
4. Maximum values for leaf: $6 - 1 = 5$

5. Leaf underflows when: values < 3
6. Internal underflows when: children < 3

Problem 2: For n = 7

1. Internal min/max children: $\lceil 7/2 \rceil = 4$ to 7
2. Leaf min/max values: $\lceil 6/2 \rceil = 3$ to 6
3. If leaf has 2 values: YES, underflowing (needs ≥ 3)
4. If internal has 3 children: YES, underflowing (needs ≥ 4)

9.2 Exercise 2: B+-Tree Insert Operations

Practice - B+-Tree Insertion (n

Given the following B+-tree with degree $n = 5$:

```
Root:  [17, 24]
Children:
[2*, 3*, 5*, 7*] | [14*, 16*] | [19*, 20*, 22*] | [24*, 27*, 29*] | [33*, 34*,
38*, 39*]
```

Tasks:

1. Insert 6*. Draw the resulting tree.
2. Starting from the original tree, insert 25*. Draw the resulting tree.
3. Starting from the original tree, insert 18*. Draw the resulting tree.

Remember:

- For $n = 5$: Leaf nodes hold 2-4 values, Internal nodes have 3-5 children
- When splitting a leaf: **Copy up** the middle key
- When splitting an internal node: **Push up** the middle key

Solutions:
Task 1: Insert 6*

- Navigate to leaf [2*, 3*, 5*, 7*]
- Insert 6*: [2*, 3*, 5*, 6*, 7*] - OVERFLOW! (5 values, max is 4)
- Split: [2*, 3*] | [5*, 6*, 7*]
- **Copy up** middle key 5 to parent
- Parent becomes: [5, 17, 24]
- **Result:** Tree has one more leaf, parent updated

Task 2: Insert 25*

- Navigate to leaf [24*, 27*, 29*]

- Insert 25*: [24*, 25*, 27*, 29*] - OVERFLOW!
- Split: [24*, 25*] | [27*, 29*]
- **Copy up** 27 to parent
- Parent [17, 24] becomes [17, 24, 27] - no overflow
- **Result:** Parent: [17, 24, 27] with updated leaves

Task 3: Insert 18*

- Navigate to leaf [19*, 20*, 22*]
- Insert 18*: [18*, 19*, 20*, 22*] - OVERFLOW!
- Split: [18*, 19*] | [20*, 22*]
- **Copy up** 20 to parent
- Parent [17, 24] becomes [17, 20, 24] - no overflow
- **Result:** Parent: [17, 20, 24] with updated leaves

9.3 Exercise 3: B+-Tree Delete Operations

Practice - B+-Tree Deletion (n

Given the following B+-tree with degree $n = 4$:

```
Root:  [17]
Children:
[5] | [24, 30]
[2*, 3*] | [5*, 7*, 8*] | [14*, 16*] | [19*, 20*, 22*] | [24*, 27*, 29*] | [33*,
34*, 38*, 39*]
```

Tasks:

1. Delete 8*. Should you redistribute or merge? Draw the result.
2. Starting from the original tree, delete 22*. Draw the result.
3. Starting from the original tree, delete 19* then 20*. Draw the final result.

Decision criteria:

- If sibling has $> \lceil (n-1)/2 \rceil$ values: **Redistribute**
- If sibling has exactly $\lceil (n-1)/2 \rceil$ values: **Merge**
- For $n = 4$: Minimum is 2 values per leaf

Solutions:
Task 1: Delete 8*

- Leaf [5*, 7*, 8*] becomes [5*, 7*] - no underflow (2 values OK)
- **Action: None needed**

Task 2: Delete 22*

- Leaf [19*, 20*, 22*] becomes [19*, 20*] - no underflow
- **Action: None needed**

Task 3: Delete 19* then 20*

- Delete 19*: [19*, 20*, 22*] → [20*, 22*] (OK)
- Delete 20*: [20*, 22*] → [22*] - UNDERFLOW! (needs ≥ 2)
- Check right sibling [24*, 27*, 29*] - has 3 values (> minimum of 2)
- **Redistribute:** Borrow 24* from right sibling
- Left leaf: [22*, 24*]
- Right leaf: [27*, 29*]
- Update parent separator from 24 to 27

9.4 Exercise 4: B+-Tree Search

Practice - B+-Tree Search Operations

Given the following B+-tree:

```
                    Root:  [20]
          Left:  [10, 16] | Right:  [30]
[1*, 5*] | [12*, 15*, 17*] | [21*, 24*] | [34*, 35*, 38*]
```

Tasks:

1. Describe the path to find 15*. Which nodes are visited?
2. Describe the path to find 25*. Which nodes are visited?
3. How would you find all values ≥ 21? Describe the process.
4. What is the maximum number of nodes visited to find any value?

Search algorithm:

- Start at root
- At each non-leaf: find smallest key $K \geq$ search value
- Follow appropriate pointer
- At leaf: search for value

Solutions:
1. Find 15*:

- Start at root [20]
- 15 < 20, go left to [10, 16]

- 15 < 16, go to second pointer (between 10 and 16)
- Reach leaf [12*, 15*, 17*]
- **Path: Root → [10, 16] → [12*, 15*, 17*]**

2. **Find 25*:**
   - Start at root [20]
   - 25 > 20, go right to [30]
   - 25 < 30, go left pointer
   - Reach leaf [21*, 24*]
   - **Path: Root → [30] → [21*, 24*]**
3. **Find all values ≥ 21:**
   - Search for 21* to reach leaf [21*, 24*]
   - Scan this leaf: 21*, 24*
   - Follow pointer to next leaf: [34*, 35*, 38*]
   - Scan remaining leaves
   - **Result: 21*, 24*, 34*, 35*, 38***
4. **Maximum nodes visited:**
   - Tree height = 3 (root → internal → leaf)
   - **Maximum: 3 nodes**

9.5 Exercise 5: Index Type Identification

Practice - Identifying Index Types

Scenario: Consider a relation `employee(id, name, dept, salary)` stored on disk.
Case 1: File is sorted by `id`. There is an index on `id` with an entry for every record.

- Is this a primary or secondary index?
- Is this a dense or sparse index?

Case 2: File is sorted by `id`. There is an index on `id` with entries only for some records.

- Is this a primary or secondary index?
- Is this a dense or sparse index?
- Can this work? Why or why not?

Case 3: File is sorted by `id`. There is an index on `dept` with an entry for every distinct dept value.

- Is this a primary or secondary index?

- Is this a dense or sparse index?
- Must this be dense? Why or why not?

Case 4: File is sorted by `dept`. There is an index on `salary`.

- Is this a primary or secondary index?
- Can this index be sparse?

Solutions:
Case 1:

- **Primary index** (file sorted by id)
- **Dense index** (entry for every record)

Case 2:

- **Primary index** (file sorted by id)
- **Sparse index** (entries only for some records)
- **YES, can work** - sparse indexes are allowed for primary indexes

Case 3:

- **Secondary index** (file NOT sorted by dept)
- **Dense index**
- **MUST be dense** - secondary indexes cannot be sparse

Case 4:

- **Secondary index** (file sorted by dept, not salary)
- **NO, cannot be sparse** - secondary indexes must be dense

9.6 Exercise 6: Hash Function Application

Practice - Hashing

Given:

- Hash function: $h(x) = x \mod 7$ (for numeric keys)
- Hash function: $h(s) = \text{length}(s) \mod 7$ (for string keys)
- 7 buckets numbered 0-6

Problem 1: Hash the following employee IDs:

- 101, 203, 305, 407, 509, 611, 713
- Which bucket does each ID map to?
- Identify any collisions

Problem 2: Hash the following department names:

- "CS", "Math", "Physics", "Biology", "History", "Music"
- Which bucket does each name map to?
- Identify any collisions

Problem 3:

- If bucket 3 can hold 2 records, what happens when a third record hashes to bucket 3?
- Draw the overflow chain after inserting all records that hash to bucket 3

Solutions:
Problem 1: Numeric keys with $h(x) = x \mod 7$

- $101 \mod 7 = 3 \rightarrow$ Bucket 3
- $203 \mod 7 = 0 \rightarrow$ Bucket 0
- $305 \mod 7 = 4 \rightarrow$ Bucket 4
- $407 \mod 7 = 1 \rightarrow$ Bucket 1
- $509 \mod 7 = 5 \rightarrow$ Bucket 5
- $611 \mod 7 = 2 \rightarrow$ Bucket 2
- $713 \mod 7 = 6 \rightarrow$ Bucket 6
- **No collisions!** Each ID maps to unique bucket

Problem 2: String keys with $h(s) = \textbf{length}(s) \mod 7$

- "CS" (length 2): $2 \mod 7 = 2 \rightarrow$ Bucket 2
- "Math" (length 4): $4 \mod 7 = 4 \rightarrow$ Bucket 4
- "Physics" (length 7): $7 \mod 7 = 0 \rightarrow$ Bucket 0
- "Biology" (length 7): $7 \mod 7 = 0 \rightarrow$ Bucket 0 **COLLISION!**
- "History" (length 7): $7 \mod 7 = 0 \rightarrow$ Bucket 0 **COLLISION!**
- "Music" (length 5): $5 \mod 7 = 5 \rightarrow$ Bucket 5
- **Collisions: Physics, Biology, History all hash to bucket 0**

Problem 3: Overflow chain

- Bucket 3 holds: 101 (first record)
- 203 doesn't hash to 3, but if another record did:
- Third record $\rightarrow$ overflow bucket linked to bucket 3
- **Structure:** Bucket 3 $\rightarrow$ Overflow Bucket $\rightarrow$ (more if needed)

9.7 Exercise 7: File Organization Comparison

Practice - Choosing File Organization

Scenario: Design storage for a student records database.
Access patterns:

- A1: Frequent lookups by student ID (exact match)
- A2: Frequent range queries by enrollment date
- A3: Sequential processing of all records
- A4: Frequent insertions of new students

Questions:

1. Which file organization is best for A1 only? (Heap, Sequential, B+-tree, Hash)
2. Which file organization is best for A2 only?
3. Which file organization is best for A3 only?
4. Which file organization is best for A4 only?
5. If you need to support all access patterns, which would you choose? Why?
6. What additional indexes might you add?

Trade-offs to consider:

Organization	Point Query	Range Query	Sequential	Insert
Heap	Slow (scan)	Slow (scan)	Fast	Very Fast
Sequential	Fast (if sorted key)	Fast	Fast	Slow (reorg)
B+-tree	Fast	Fast	Fast	Fast
Hash	Very Fast	Slow	Slow	Fast

Solutions:
1. Best for A1 (exact match lookup): Hash - Very fast exact match
2. Best for A2 (range queries): B+-tree or Sequential - Both support range
3. Best for A3 (sequential processing): Sequential or Heap - Fast scan
4. Best for A4 (frequent insertions): Heap or Hash - Very fast insert
5. Support all patterns: B+-tree

- Good for exact match (A1)
- Excellent for range queries (A2)
- Good for sequential (A3)
- Good for insertions (A4)
- Most balanced trade-off

6. Additional indexes:

- Primary: B+-tree on student_id (for A1, A3, A4)
- Secondary: B+-tree on enrollment_date (for A2)
- Optional: Hash index on student_id (for very fast A1)

9.8 Exercise 8: B+-Tree Split/Merge Decision

Practice - Split and Merge Decisions

For each scenario, decide what operation to perform:
Scenario 1: $n = 5$, inserting into leaf with values [10, 20, 30, 40]

- Current count: 4 values
- Maximum allowed: $n - 1 = 4$ values
- Action needed?

Scenario 2: $n = 5$, inserting into leaf with values [10, 20]

- Current count: 2 values
- Maximum allowed: 4 values
- Action needed?

Scenario 3: $n = 6$, deleting from leaf with values [10, 20, 30], sibling has [40, 50, 60]

- After deletion: 2 values
- Minimum required: $\lceil (6 - 1)/2 \rceil = 3$ values
- Sibling has: 3 values (minimum)
- Action: Redistribute or Merge?

Scenario 4: $n = 6$, deleting from leaf with values [10, 20, 30], sibling has [40, 50, 60, 70]

- After deletion: 2 values (underflow)
- Sibling has: 4 values ($>$ minimum of 3)
- Action: Redistribute or Merge?

Solutions:
Scenario 1:

- After insertion: 5 values
- Max allowed: 4 values
- **Action: SPLIT** (overflow)

Scenario 2:

- After insertion: 3 values
- Max allowed: 4 values
- **Action: None** (no overflow)

Scenario 3:

- After deletion: 2 values $<$ minimum (3)
- Sibling: 3 values $=$ minimum (cannot spare)

- **Action: MERGE**

Scenario 4:

- After deletion: 2 values < minimum (3)
- Sibling: 4 values > minimum (can spare one)
- **Action: REDISTRIBUTE**

9.9 Exercise 9: Storage Hierarchy

Practice - Understanding Storage Trade-offs

Problem 1: Rank the following in order of access speed (fastest to slowest):

- Magnetic Disk, Main Memory (RAM), Cache, Flash SSD, Tape

Problem 2: Rank the same storage types by cost per GB (cheapest to most expensive).
Problem 3: Which storage types are volatile? Which are non-volatile?
Problem 4: Given these requirements, choose the best storage tier:

1. Frequently accessed user session data (needs to persist across restarts)
2. Database index that is queried millions of times per second
3. Archive of 5-year-old transaction logs (rarely accessed)
4. Working set for current database transactions

Solutions:
Problem 1: Speed ranking (fastest to slowest)

1. Cache
2. Main Memory (RAM)
3. Flash SSD
4. Magnetic Disk
5. Tape

Problem 2: Cost per GB (cheapest to most expensive)

1. Tape
2. Magnetic Disk
3. Flash SSD
4. Main Memory (RAM)
5. Cache

Problem 3: Volatile vs Non-volatile

- **Volatile:** Cache, Main Memory (RAM)

- **Non-volatile:** Flash SSD, Magnetic Disk, Tape

Problem 4: Best storage tier

1. User session data: **Flash SSD** (fast + persistent)
2. Database index: **Main Memory (RAM)** (very fast access)
3. Old transaction logs: **Tape** (cheap archival storage)
4. Working set: **Main Memory (RAM)** (active transactions need speed)

9.10 Exercise 10: Integrated Problem

Practice - Comprehensive B+-Tree Problem

Given: Empty B+-tree with degree $n = 4$
Task: Insert the following values in order: 10, 20, 30, 5, 15, 25, 35, 12
Requirements:

1. Draw the tree after each insertion
2. Identify when splits occur
3. Label each split as leaf split or internal split
4. Indicate whether keys are copied up or pushed up
5. Show the final tree structure

Then: Delete values 12, 15, 30 in that order from the final tree
Requirements:

1. Draw the tree after each deletion
2. Identify when redistribution or merging occurs
3. Explain your decision (why redistribute vs merge)
4. Show the final tree structure

Remember for $n = 4$:

- Leaf nodes: 2-3 values
- Internal nodes: 2-4 children
- Underflow: leaf < 2 values, internal < 2 children

Solution:
Insertions:

1. Insert 10: [10*]
2. Insert 20: [10*, 20*]
3. Insert 30: [10*, 20*, 30*]
4. Insert 5: [5*, 10*, 20*, 30*] - OVERFLOW! Split:

5*, 10* | [20*, 30*], copy up 20 → Root: [20]

5. Insert 15: Left leaf [5*, 10*] → [5*, 10*, 15*]
6. Insert 25: Right leaf [20*, 30*] → [20*, 25*, 30*]
7. Insert 35: Right leaf [20*, 25*, 30*] → [20*, 25*, 30*, 35*] - OVERFLOW!
   - Split: [20*, 25*] | [30*, 35*], copy up 30
   - Root: [20, 30]
8. Insert 12: Left leaf [5*, 10*, 15*] → [5*, 10*, 12*, 15*] - OVERFLOW!
   - Split: [5*, 10*] | [12*, 15*], copy up 12
   - Root: [12, 20, 30]

Final tree after all insertions:

- Root: [12, 20, 30]
- Leaves: [5*, 10*] | [12*, 15*] | [20*, 25*] | [30*, 35*]

Deletions:

1. Delete 12: [12*, 15*] → [15*] - UNDERFLOW!
   - Left sibling [5*, 10*] has 2 (minimum) → MERGE
   - Result: [5*, 10*, 15*]
   - Remove 12 from root → [20, 30]
2. Delete 15: [5*, 10*, 15*] → [5*, 10*] - no underflow
3. Delete 30: [30*, 35*] → [35*] - UNDERFLOW!
   - Left sibling [20*, 25*] has 2 (minimum) → MERGE
   - Result: [20*, 25*, 35*]
   - Remove 30 from root → [20]

Final tree:

- Root: [20]
- Leaves: [5*, 10*] | [20*, 25*, 35*]

9.11 Quick Self-Check Questions

Rapid Review Questions

Answer True or False:

1. In a B+-tree, all leaf nodes are at the same level (T/F)
2. Sparse indexes can be used for secondary indexes (T/F)
3. When splitting a leaf node, the middle key is pushed up to the parent (T/F)

4. Hash indexes are efficient for range queries (T/F)
5. A primary index must be on the primary key (T/F)
6. Flash memory is volatile storage (T/F)
7. Sequential file organization is good for frequent insertions (T/F)
8. In a B+-tree of degree 5, a leaf can have 1 value (T/F)
9. Dense indexes have an entry for every search-key value (T/F)
10. The root of a B+-tree must have at least 2 children (T/F)

Answers: 1-T, 2-F, 3-F (copied up), 4-F, 5-F, 6-F, 7-F, 8-F (min is 2), 9-T, 10-T (if not a leaf)

10 Visual Reference - B+-Tree Operations Summary

B+-Tree Operations Cheat Sheet

INSERT Algorithm:

1. Navigate to correct leaf
2. Insert value in sorted order
3. If leaf has $\leq n - 1$ values: DONE
4. If leaf has n values: SPLIT
   - Divide into two nodes
   - **COPY UP** smallest key from right node
   - Insert copied key into parent
   - If parent overflows, split it (**PUSH UP** middle key)

DELETE Algorithm:

1. Navigate to leaf and delete value
2. If leaf has $\geq \lceil (n-1)/2 \rceil$ values: DONE
3. If leaf underflows:
   - Check sibling
   - If sibling has extra: **REDISTRIBUTE**
   - If sibling at minimum: **MERGE**

Key Difference:

- Leaf split: Middle key **copied up** (stays in leaf)
- Internal split: Middle key **pushed up** (moves to parent)